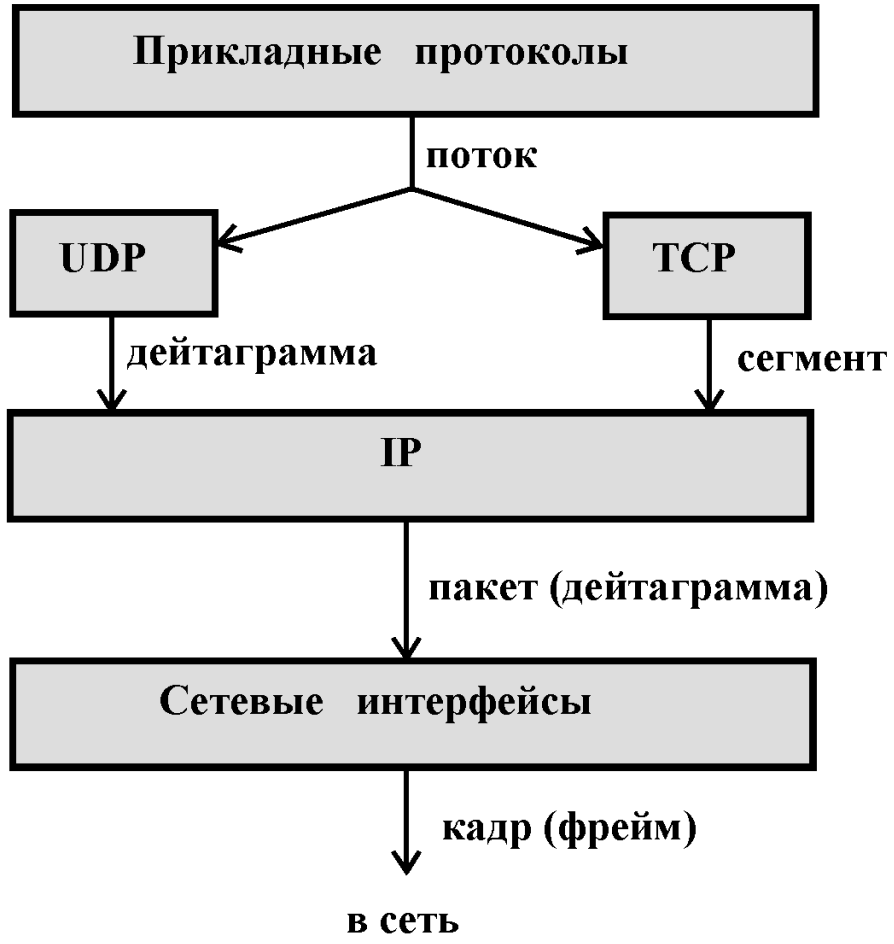
Лекция 12. Транспортный уровень

Цель транспортного уровня

– обеспечение приложениям и верхним уровням модели OSI передачи данных той степени надежности, которая им требуется.

Транспортный уровень образует границу между поставщиком сетевых услуг и пользователем транспортных услуг.



Порты

Протоколы транспортного уровня имеют свои системные очереди из данных:

- поступающих к приложению из сети;
- поступающих этими приложениями в сеть.

Данные системные очереди называются **портами**. Входная и выходная очереди одного приложения рассматриваются как один порт.

Порты идентифицируются номерами

- от 0 до 1023 – уникальные порты в пределах сети Интернет
- от 1024 до 49151 – зарегистрированные порты
- от 49152 до 65535 – динамические (частные) порты

Сокеты

– набор идентифицирующих параметров, которые однозначно определяют прикладной процесс в пределах сети и в пределах отдельного компьютера.

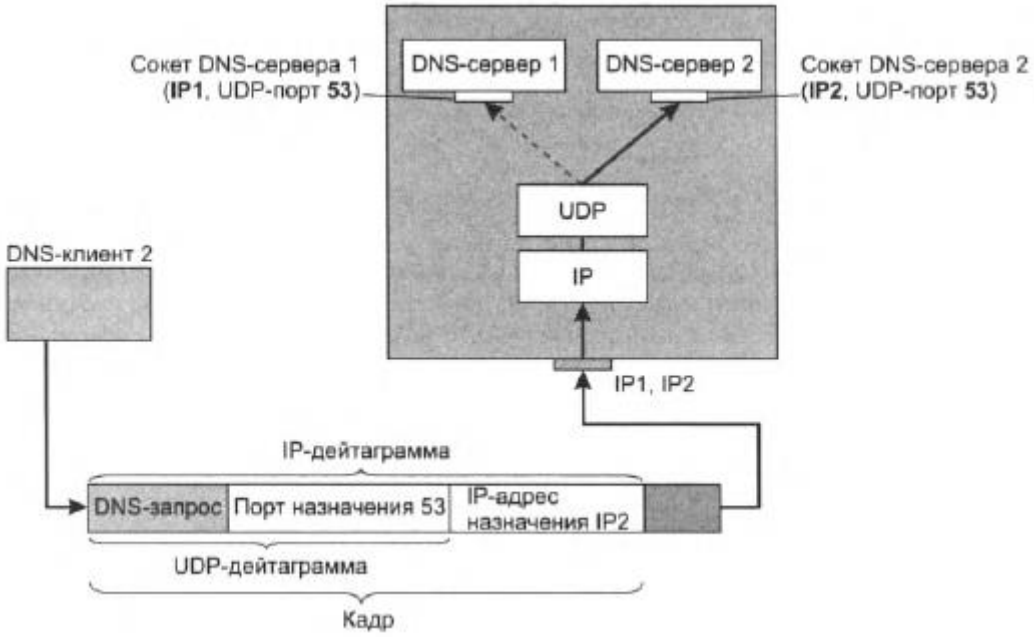
IP-адрес хоста: номер порта

Номер порта является локальным по отношению к хосту. Назначение номеров портов прикладным процессам осуществляется:

- **Централизованно**, если процессы предоставляют общедоступные службы Интернета. Централизованным присвоением номеров портов службам занимается организация IANA.
- **Локально**, когда разработчик приложения связывает с портом произвольно выбранный числовой идентификатор.

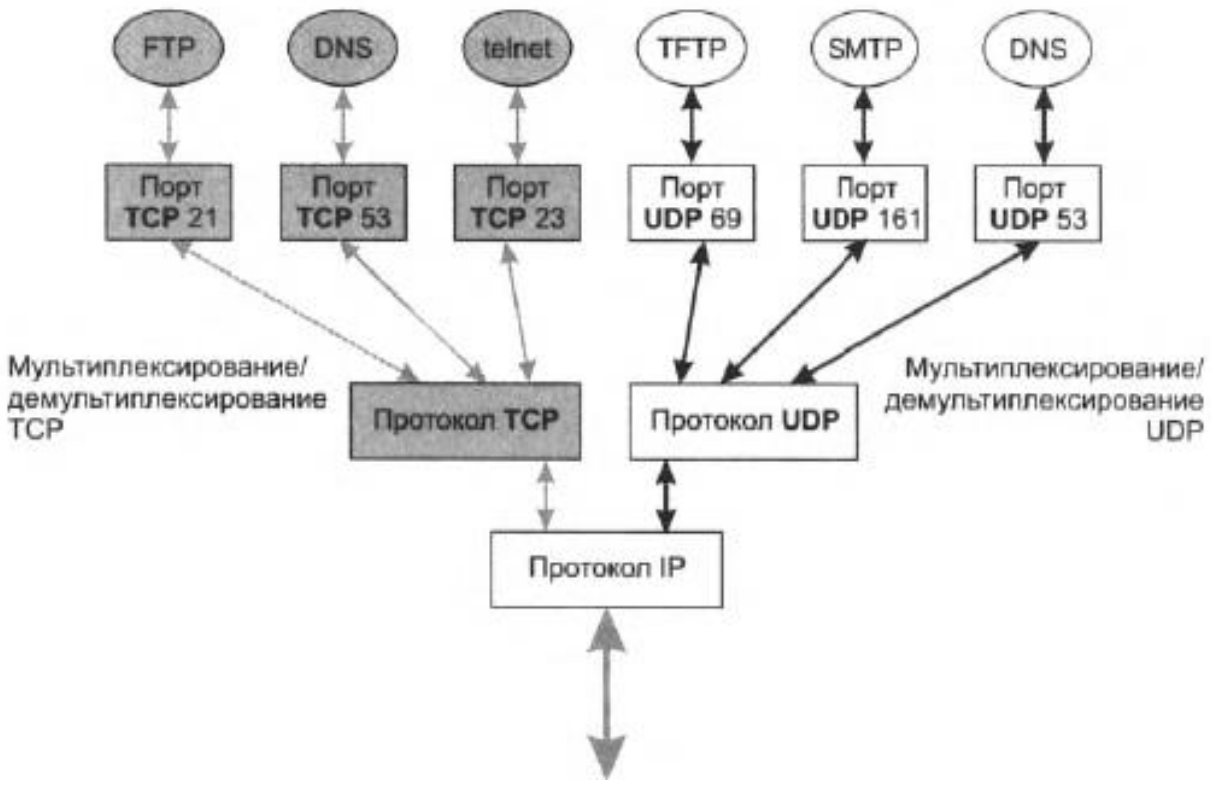
Назначение сокетов

- Стандартные назначения номеров портов уникально идентифицируют тип приложения (FTP, DNS, HTTP), однако они не могут использоваться для однозначной идентификации прикладных процессов, связанными с каждым из этих типов приложений.
- Для исключения неоднозначности в идентификации приложений, разные копии приложений связываются с разными IP-адресами.
- Для этого сетевой интерфейс компьютера, на котором выполняется несколько копий приложений, должен иметь соответствующее число IP-адресов.



Демультиплексирование и мультиплексирование на транспортном уровне

- **Демультиплексирование** – реализуемая протоколами TCP и UDP процедура распределения прикладными процессами пакетов, поступающих от сетевого уровня.
- **Мультиплексирование** – процесс передачи данных, генерируемых различными приложениями и работающих на одном оконечном узле, общему для всех них протокольному модулю IP для последующей отправки в сеть.



Протоколы TCP и UDP

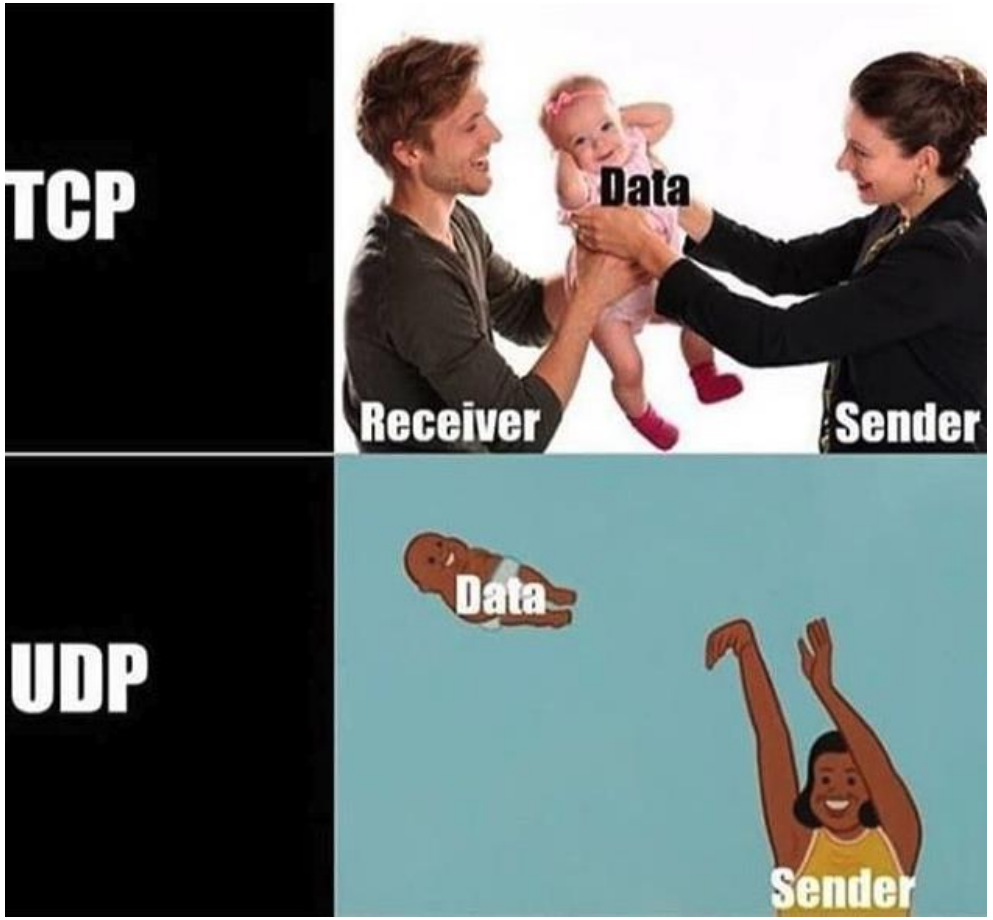
Протокол **TCP** (Transmission Control Protocol, Протокол управления передачей) – потоковый транспортный сервис с надежной доставкой.

Протокол **UDP** (User Datagram Protocol, Протокол пользовательских дейтаграмм) – сервис негарантированной доставки единичных сообщений.

Протоколы TCP и UDP

TCP

UDP



Протокол UDP

- Является датаграммным протоколом, который реализует ненадежный сервис и не гарантирует доставку пакета.
- Взаимодействует в режиме без установления логического соединения.
- Не обеспечивает правильный порядок передачи данных.
- Не имеет средств уведомления о правильности ошибочности UDP в его приеме адресатом.

Структура UDP-датаграммы и его псевдозаголовка

Заголовок

```
 0      7 8     15 16    23 24    31
+--------+--------+--------+--------+
|     Source      |   Destination   |
|      Port       |      Port       |
+--------+--------+--------+--------+
|                 |                 |
|     Length      |    Checksum     |
+--------+--------+--------+--------+
|
|          data octets ...
+---------------- ...
```

Псевдозаголовок

```
 0      7 8     15 16    23 24    31
+--------+--------+--------+--------+
|          source address           |
+--------+--------+--------+--------+
|        destination address        |
+--------+--------+--------+--------+
|  zero  |protocol|   UDP length    |
+--------+--------+--------+--------+
```

Основные функции TCP

1. **Базовая передача данных**. TCP рассматривает информацию как неструктурированный поток байтов.
2. **Обеспечение достоверности**. Обеспечивает защиту от повреждения, потери, дублирования и нарушения очередности получения данных. Для выполнения этих процессов все октеты в потоке данных сквозным образом пронумерованы в возрастающем порядке.

   Для каждого сегмента вычисляется контрольная сумма, позволяющая обнаружить повреждение.

   Квитирование – используется для упорядочивания и защищает от дублирования данных.

Основные функции TCP

3. **Разделение каналов**. Обеспечивает работу нескольких соединений одновременно
   - Каждый прикладной процесс идентифицируется номером порта.
   - Заголовок TCP-сегмента содержит номера портов процесса отправителя и процесса-получателя.
   - Сокет уникально идентифицирует прикладной процесс в Интернет.
4. **Управление соединением**. **Соединение** – это совокупность информации о состоянии потока данных, включающая сокеты, номера посланных, принятых и подтверждённых октетов, размеры окон.
5. **Обеспечение надежной передачи данных** через ненадежную составную сеть.

Структура заголовка TCP

```
 0                   1                   2                   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|          Source Port          |       Destination Port        |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                        Sequence Number                        |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                    Acknowledgment Number                      |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|  Data |           |C|E|U|A|P|R|S|F|                           |
| Offset| Rsrvd     |W|C|R|C|S|S|Y|I|            Window             |
|       |           |R|E|G|K|H|T|N|N|                           |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|           Checksum            |         Urgent Pointer        |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                           [Options]                           |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                                                               :
:                             Data                              :
:                                                               |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

Структура заголовка TCP

- **Source port** (порт отправителя)
- **Destination port** (порт получателя)
- **Sequence number** (порядковый номер) – номер первого байта данных в сегменте, определяет смещение сегмента относительно потока отправляемых данных.
- **Acknowledgment Number** (номер подтверждения) – максимальный номер байта в полученном сегменте, увеличенный на единицу.
- **Data Offset** (длина заголовка) – размер заголовка, измеряется в 32-х битных словах.
- **Rsrvd** (reserved) – зарезервировано под будущее использование.

Структура заголовка TCP. Флаги

- **CWR** – отображает, что сокращено окно перезагрузки;
- **ECE** – уведомление о перезагрузке принято;
- **URG** – срочное сообщение;
- **ACK** – квитанция на принятый сегмент;
- **PSH** – запрос на отправку сообщения без ожидания заполнения буфера;
- **RST** – запрос на сброс соединения;
- **SYN** – сообщение, используемое для синхронизации счетчиков переданных данных при установлении соединения;
- **FIN** – признак достижения передающей стороной последнего байта в потоке передаваемых данных.

Структура заголовка TCP

- **Window** (окно) - количество байтов данных, ожидаемых отправителем данного сегмента, начиная с байта, номер которого указан в поле подтверждения номера.
- **Checksum** (контрольная сумма) содержит сумму заголовка, данных и псевдозаголовка.
- **Urgent Pointer** (конец заголовка) – указывает на конец данных, которые необходимо срочно принять, несмотря на переполнение буфера.
- **Options** (параметры) – это поле имеет переменную длину и может вообще отсутствовать, используется для решения вспомогательных задач, например, для согласования максимального размера сегмента.

Установление TCP-соединения

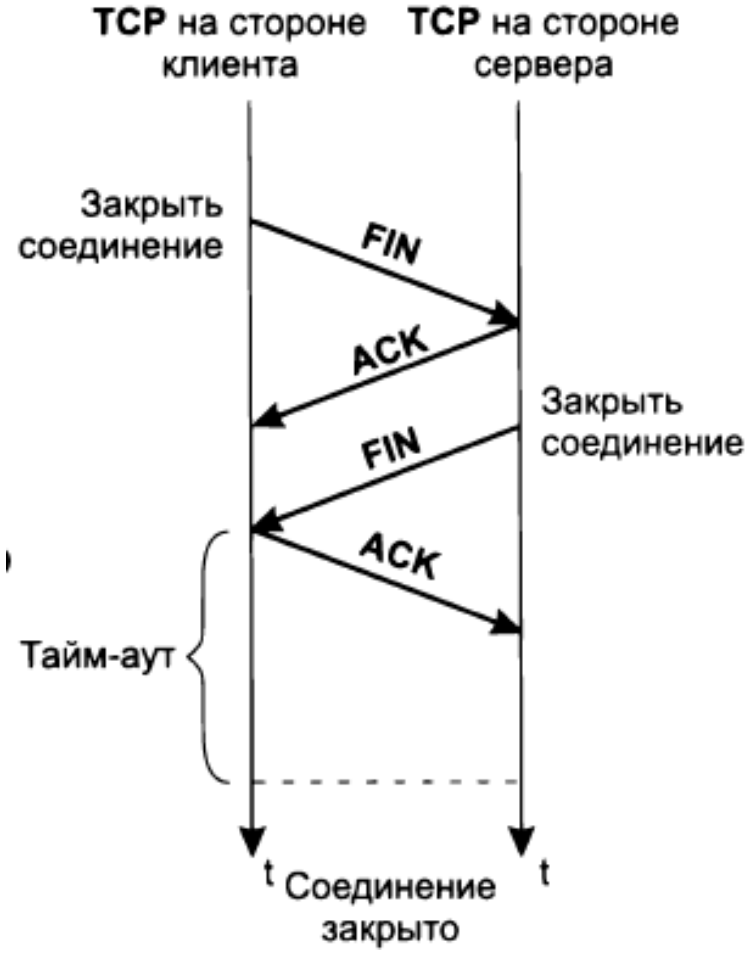
1.Клиент обращается к протоколу TCP, который в ответ на это обращение посылает сегмент-запрос на установление соединения по протоколу TCP, работающему на стороне сервера. В числе прочего в запросе содержится флаг **SYN**, установленный в 1.

2.Получив запрос, модуль TCP на стороне сервера пытается создать «инфраструктуру» для обслуживания нового клиента. Если все было получено и создано, то модуль TCP посылает клиенту сегмент с флагами **ACK и SYN**.

3.В ответ клиент посылает **ACK** и переходит в состояние логического соединения ESTABLISHED. Когда сервер получает ACK он также переходит в состояние ESTABLISHED.

Разрыв TCP-соединения

- Соединение может быть разорвано в любой момент по инициативе любой стороны.
- Для этого клиент и сервер должны обменяться сегментами FIN и ACK, в порядке, показанном на рисунке (здесь инициатор разрыва – клиент)
- Соединение считается закрытым по прошествии некоторого времени (тайм-аут), в течении которого сторона-инициатор убеждается, что ее завершающий сигнал ACK дошел нормально и не вызывал никаких «аварийных» сообщений со стороны сервера.



Квитирование в TCP

- Отправитель отсылает данные, а получатель подтверждает их получение квитанциями (ACK). Если отправитель вовремя не получает квитанции на переданные данные, то он передает их повторно.
- Если отправитель вовремя не получает квитанции на переданные данные, то он передает их повторно, выполняя запрос повторной передачи (Automatic Repeat reQuest, **ARQ**).

Методы квитирования

- **Метод простоя источника** (Stop-and-Wait)
- **Метод скользящего окна** (Sliding Window)
  - Методы, использующие окно передачи. Пример: *метод передачи с возвращением на N пакетов* (Go-Back-N)
  - Методы, использующие окно передачи и окно приема. Пример: *метод передачи с выборочным повторением* (Selective Repeat).

Метод простоя источника (Stop-and-Wait)

- В методе простоя источника отправитель передает последовательность **ненумерованных пакетов**.
- Метод требует, чтобы отправитель дожидался от получателя квитанции и только после этого посылал следующий пакет.
- С переданным пакетом отправитель связывает **таймер**.
- Если в течение тайм-аута квитанция не пришла, то пакет (или квитанция на него) считается утерянным или искаженным и его передача повторяется.

отправка данных -> ожидание подтверждения (ACK) -> отправка данных

Метод скользящего окна (Sliding Window)

– это метод управления передачей данных, при котором отправитель может отправить несколько сегментов в непрерывном, не дожидаясь подтверждений для каждого.

Основная цель метода – поддерживать в сети постоянный «поток» кадров для эффективного использования канала.

- **Окно отправителя** – диапазон допустимых номеров кадров, которые можно отправить.
- **Окно получателя** – диапазон номеров кадров, которые приёмник готов принять.

Основные частные случаи методов скользящего окна

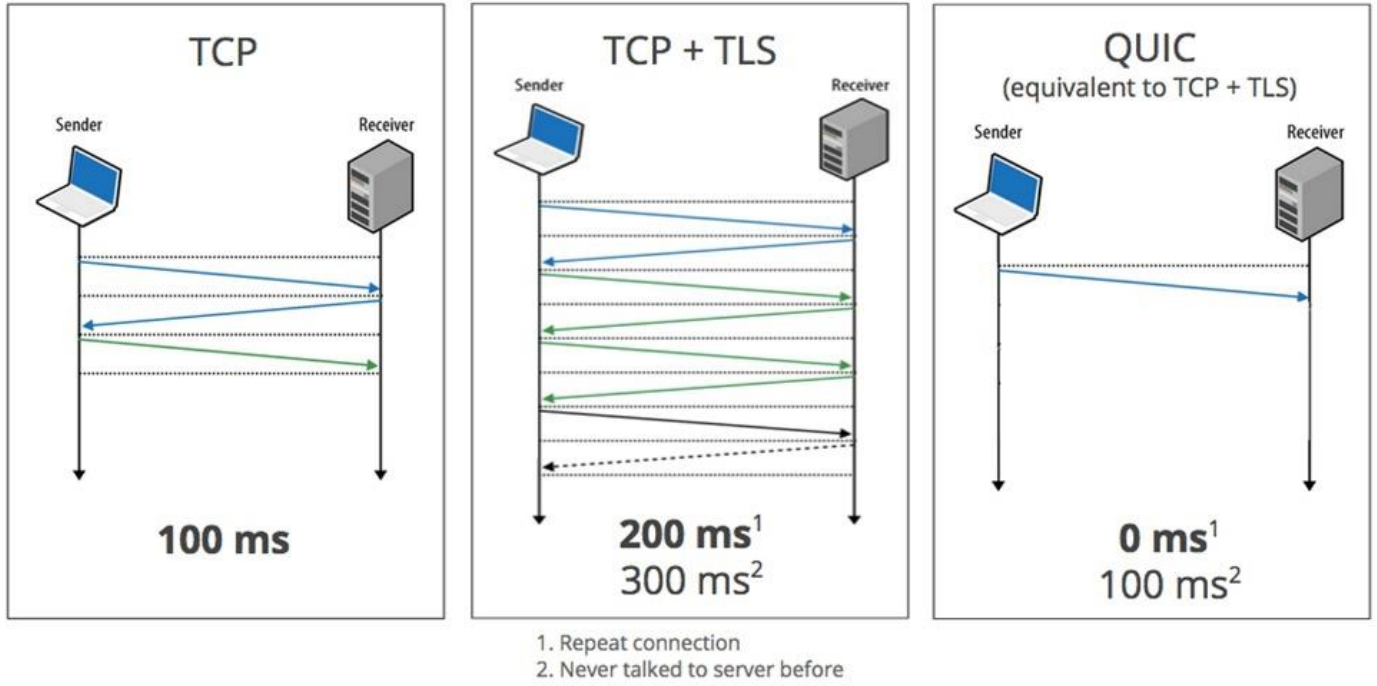
- **Метод передачи с возвращением на N пакетов** (Go-Back-N) – при потере одного кадра отправитель повторно передаёт все кадры начиная с потерянного, даже если последующие были доставлены успешно.
- **Метод передачи с выборочным повторением** (Selective Repeat) – при потере кадра отправитель переотправляет только те кадры, которые действительно потеряны, а приёмник буферизует полученные вышедшие из порядка кадры.

Проблемы TCP

- **Долгая установка соединения** особенно при дополнительном использовании шифрования при помощи TLS.
- Поддерживает только **один поток передачи данных**. Если в процессе передачи данных будет утерян сегмент, то весь поток остановится, пока сегмент не будет доставлен повторно
- **Необязательно шифрование**.
- **Привязка к IP-адресу**. При смене сети происходит разрыв TCP-соединения (например, при переходе с Wi-Fi на мобильную связь).

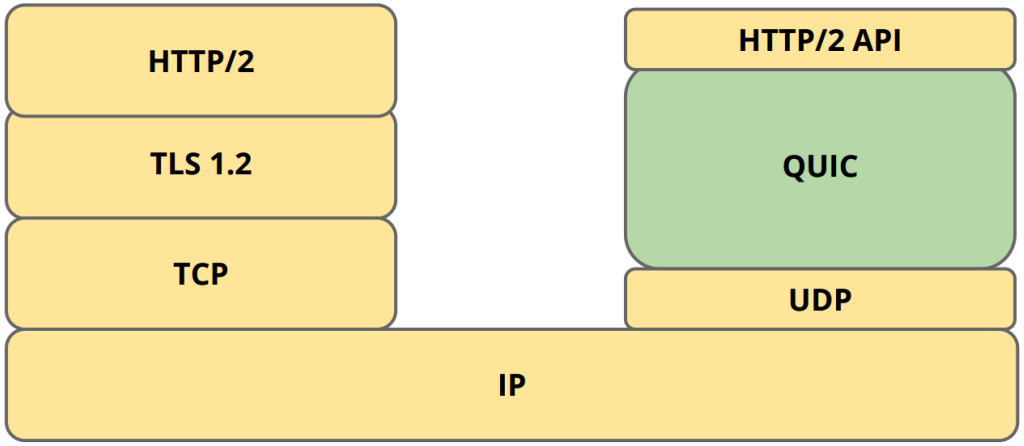
QUIC vs TCP

- Оптимизирован процесс установки соединения
- Используются независимые потоки передачи данных
- Встроенное шифрование TLS
- Использование Connection ID, который независим от IP или порта



Протокол QUIC (Quick UDP Internet Connections)

- Разработан Google
- Описан в стандарте RFC 9000
- Работает на основе UDP
- Уже используется практически всеми браузерами и различными сервисами
- HTTP/3 использует QUIC (вместо TCP)



Прочие протоколы транспортного уровня

- DCCP (Datagram Congestion Control Protocol)
- SCTP (Stream Control Transmission Protocol)
- UDT (UDP-based Data Transfer)
- RUDP (Reliable UDP)

Спасибо за внимание!